

Studentu Duomenų Apžvalgos Sistema

2.0

Sugeneruota Doxygen 1.17.0

1 StudentuDuom - studentu duomenu apdorojimo aplikacija	1
1.1 Aprasas	1
1.2 Instaliavimas / paleidimas	1
1.2.0.1 Reikalavimai	1
1.2.0.2 Kompiliavimo zingsniai	1
1.2.0.3 Sukuriame build folder	1
1.2.0.4 Konfiguruoti	1
1.2.0.5 Kompiliuoti	1
1.2.0.6 Paleisti (Linux/macOS)	1
1.2.0.7 Paleisti (Windows OS)	2
1.3 Versija v1.5	2
1.3.1 Pakeitimai	2
1.3.2 UML klasiu hierarchijos diagrama	2
1.3.3 Zmogus - abstrakti klase + rankinis v1.2 pagristas testas	2
1.4 Testavimas - v0.2	2
1.4.1 Tyrimas 1	2
1.4.1.1 1. Anksciau sugeneruotu failu nuskaitymas:	2
1.5 Testavimas - v0.4	3
1.5.1 Tyrimas 1	3
1.5.1.1 1. Failu generavimas ir uždarymas:	3
1.5.1.2 Tyrimo 1 output ekrane:	3
1.5.2	3
1.5.3 Tyrimas 2	3
1.5.3.1 1. Duomenu nuskaitymas is anksciau sugeneruotu failu:	3
1.5.3.2 2. Studentu rusiavimas i "gerus" ir "blogus" pagal galutini vidurki (≥ 5.0 - geras, kitaip - blogas):	4
1.5.3.3 3. "Geru" ir "blogu" studentu atitinkamu failu generavimas:	4
1.5.3.4 4. Visos programos laikas:	4
1.5.3.5 Tyrimo 2 output ekrane:	4
1.5.4	4
1.6 Testavimas - v1.0 subrelease	4
1.6.1 Tyrimas 1 - Aprasas	5
1.6.2 Aktualus testavimo sistemos parametrai:	5
1.6.2.1 CPU	5
1.6.2.2 RAM	5
1.6.2.3 SSD	5
1.6.3 Tyrimas 1 - Rezultatai	5
1.6.3.1 STD::VECTOR	5
1.6.3.2 STD::LIST	6
1.6.3.3 STD::DEQUE	7
1.6.4 Tyrimo 1 rezultatu interpretacija	7
1.7 Testavimas - v1.0 optimizacija	7

1.7.1 Tyrimas 2 - Aprasas	7
1.7.2 Tyrimas 2 - Rezultatai	8
1.7.3 Tyrimo 2 rezultatu interpretacija	8
2 Hierarchijos Indeksas	9
2.1 Klasių hierarchija	9
3 Klasės Indeksas	11
3.1 Klasės	11
4 Failo Indeksas	13
4.1 Failai	13
5 Klasės Dokumentacija	15
5.1 Studentas Klasė	15
5.1.1 Metodų Dokumentacija	16
5.1.1.1 read()	16
5.2 Timer Klasė	16
5.3 Zmogus Klasė	16
6 Failo Dokumentacija	19
6.1 functions.h	19
6.2 mylib.h	19
6.3 Studentas.h	19
6.4 timer.h	24
Rodyklė	25

skyrius 1

StudentuDuom - studentu duomenų apdorojimo aplikacija

1.1 Aprasas

1.2 Instaliavimas / paleidimas

1.2.0.1 Reikalavimai

- CMake (versija 3.10 arba naujesnė)
- C++17 palaikantis kompiliatorius (g++, clang++ arba MSVC)

1.2.0.2 Kompiliavimo žingsniai

1.2.0.3 Sukuriame build folder

```
mkdir build
```

```
cd build
```

1.2.0.4 Konfiguruoti

```
cmake ..
```

1.2.0.5 Kompiliuoti

```
cmake --build .
```

1.2.0.6 Paleisti (Linux/macOS)

```
./bin/student_program
```

1.2.0.7 Paleisti (Windows OS)

.\bin\Debug\student_program.exe

1.3 Versija v1.5

1.3.1 Pakeitimai

- Klase [Studentas](#) pakeista į Studentas::Zmogus, prideta abstrakti klase [Zmogus](#).
 - Klasių hierarchija aprasyta UML diagramoje žemiau.
- Atliktas rankinis testavimas, vos neidentiskas į v1.2, su skirtumu, kad iliustruota, jog [Zmogus](#) yra abstrakti klase.
 - Demonstracija pateikta nuotraukoje žemiau.
- v1.2 logika ir veikimo principai išlikę.

1.3.2 UML klasių hierarchijos diagrama

1.3.3 Zmogus - abstrakti klase + rankinis v1.2 pagristas testas

1.4 Testavimas - v0.2

Tyrimas buvo atliktas su Visual Studio 2022 /O2 optimizacijos nustatymais.
Tyrimo 1 failai buvo ištrinti prieš kiekvieną bandymą.
Testavimo metu testavimo sistemoje nebuvo įjungtos jokios kitos programos.

1.4.1 Tyrimas 1

1.4.1.1 1. Anksciau sugeneruotu failu nuskaitymas:

	1000	100000	1000000
1.	0.0478s	0.619s	2.965s

	1000	100000	1000000
2.	0.0489s	0.616s	2.894s
3.	0.0478s	0.647s	2.983s
4.	0.0479s	0.593s	2.979s
5.	0.0498s	0.640s	2.966s
Vid.	0.04844s	0.623s	2.9574s

1.5 Testavimas - v0.4

Tyrimai buvo atlikti su Visual Studio 2022 /O2 optimizacijos nustatymais.

Tyrimo 1 ir tyrimo 2 failai buvo istrinti prieš kiekviena bandyma, išskyrus tyrimo 2 failus, su kuriais buvo testuojamas skaitymo greitis (Tyrimas 2.1).

Testavimo metu testavimo sistemoje nebuvo įjungtos jokios kitos programos.

1.5.1 Tyrimas 1

1.5.1.1 1. Failu generavimas ir uždarymas:

	1000	10000	100000	1000000	10000000
1.	0.003s	0.03s	0.31s	3.29s	31.4s
2.	0.004s	0.03s	0.34s	3.36s	31.9s
3.	0.004s	0.03s	0.34s	3.3s	31.5s
4.	0.003s	0.04s	0.35s	3.32s	31.7s
Vid.	0.0035s	0.0325s	0.335s	3.318s	31.625s

1.5.1.2 Tyrimo 1 output ekrane:

1.5.2

Kitas nuotraukas galima rasti repozitorijos assets aplanke (v0.4/Assets/...).

1.5.3 Tyrimas 2

1.5.3.1 1. Duomenų nuskaitymas iš anksciau sugeneruotu failu:

	1000	10000	100000	1000000	10000000
1.	0.005s	0.04s	0.46s	4.62s	47.37s
2.	0.007s	0.04s	0.47s	4.78s	47.66s
3.	0.005s	0.05s	0.47s	4.75s	47.72s
4.	0.006s	0.05s	0.47s	4.7s	47.46s

	1000	10000	100000	1000000	10000000
Vid.	0.00575s	0.045s	0.4675s	4.7125s	47.5525s

1.5.3.2 2. Studentu rusiavimas i "gerus" ir "blogus" pagal galutini vidurki (≥ 5.0 - geras, kitaip - blogas):

	1000	10000	100000	1000000	10000000
1.	0.0004s	0.0015s	0.01s	0.11s	1.25s
2.	0.0002s	0.0013s	0.009s	0.11s	1.19s
3.	0.0003s	0.006s	0.01s	0.11s	1.28s
4.	0.0003s	0.0012s	0.01s	0.11s	1.29s
Vid.	0.003s	0.0025s	0.00975s	0.11s	1.2525s

1.5.3.3 3. "Geru" ir "blogu" studentu atitinkamu failu generavimas:

	1000	10000	100000	1000000	10000000
1.	0.004s	0.03s	0.3s	4.03s	30.72s
2.	0.005s	0.03s	0.31s	3.18s	30.97s
3.	0.004s	0.03s	0.31s	3.14s	31.12s
4.	0.004s	0.03s	0.32s	3.1s	31.04s
Vid.	0.017s	0.03s	0.31s	3.3625s	30.9625s

1.5.3.4 4. Visos programos laikas:

	1000	10000	100000	1000000	10000000
1.	0.026s	0.09s	0.78s	8.83s	79.75s
2.	0.013s	0.09s	0.8s	8.12s	80.25s
3.	0.011s	0.099s	0.81s	8.04s	80.56s
4.	0.011s	0.088s	0.81s	7.95s	80.2s
Vid.	0.01525s	0.09175s	0.8s	8.235s	80.19s

1.5.3.5 Tyrimo 2 output ekrane:

1.5.4 ``

Kitas nuotraukas galima rasti repozitorijos assets aplanke (v0.4/Assets/...).

1.6 Testavimas - v1.0 subrelease

Tyrimai buvo atlikti su Visual Studio 2022 /O2 optimizacijos nustatymais.

Tyrimo 1 failai buvo sukurti viena karta prieš bandymu pradzia.

Testavimo metu testavimo sistemoje nebuvo ijungtos jokios kitos programos.

1.6.1 Tyrimas 1 - Aprasas

Atliktas tyrimas su trejais skirtingais STL konteineriais (Vektoriai, Sarasai (list) ir Deque).

Atlikti 3 bandymai kiekvienam konteineriui ir apskaičiuotas vidurkis, kuris pateikiamas kuo tikslesnis (daugiausia 5 simboliai po kablelio).

Siame tyrime buvo ismatuota minetu konteineriu sparta, programai:

1. skaitant duomenis is failo,
2. rusiuojant duomenis didejimo tvarka pagal galutini vidurki (sort funkcija),
3. skirstant duomenis pagal galutini vidurki, sukuriant du naujus tokio pat tipo konteinerius ir naudojant move() funkcija, kad perkelti duomenis is originalaus konteinerio.

Rezultatai pateikiami sekundemis, suapvalinti (stengiamasi nevirsyti 3 skaitmenu po kablelio, taciau yra labai mazu duomenu).

1.6.2 Aktualus testavimo sistemos parametrai:

1.6.2.1 CPU

Specification: 12th Gen Intel Core i7-12650H

Core count: 10

Thread count: 16

Hyperthreading: Not supported

1.6.2.2 RAM

Specification: DDR5-4800 (2400 MHz)

Memory slots used: 2 out of 2

Total memory: 16 GB

1.6.2.3 SSD

Specification: NVMe SAMSUNG MZVLQ1T0HBLB-00B00

Read/Write speed: 2300/1350 (MB/s)

Total capacity: 953 GB

1.6.3 Tyrimas 1 - Rezultatai

1.6.3.1 STD::VECTOR

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
1 000	Nuskaitymas	0.005	0.0048	0.0048	0.0049

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
	Rusiavimas (sort)	-	-	-	-
	Skirstymas	-	-	-	-
	Viskas	-	-	-	-
10 000	Nuskaitymas	0.045	0.047	0.047	0.046
	Rusiavimas (sort)	0.001	0.001	0.001	0.001
	Skirstymas	0.002	0.002	0.002	0.002
	Viskas	0.048	0.05	0.05	0.05
100 000	Nuskaitymas	0.47	0.48	0.46	0.47
	Rusiavimas (sort)	0.01	0.011	0.010	0.010
	Skirstymas	0.018	0.018	0.019	0.018
	Viskas	0.50	0.51	0.49	0.50
1 000 000	Nuskaitymas	4.57	4.60	4.63	4.60
	Rusiavimas (sort)	0.1180	0.117	0.12	0.12
	Skirstymas	0.19	0.19	0.19	0.19
	Viskas	4.88	4.90	4.94	4.91
10 000 000	Nuskaitymas	46.49	46.35	46.66	46.50
	Rusiavimas (sort)	1.11	1.12	1.19	1.14
	Skirstymas	2.28	2.45	2.38	2.37
	Viskas	49.89	49.92	50.26	50.01

1.6.3.2 STD::LIST

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
1 000	Nuskaitymas	0.005	0.006	0.005	0.005
	Rusiavimas (sort)	-	-	-	-
	Skirstymas	-	-	-	-
	Viskas	0.006	0.006	0.006	0.006
10 000	Nuskaitymas	0.047	0.046	0.044	0.046
	Rusiavimas (sort)	0.001	0.001	0.001	0.001
	Skirstymas	0.003	0.003	0.003	0.003
	Viskas	0.051	0.051	0.048	0.050
100 000	Nuskaitymas	0.450	0.459	0.462	0.457
	Rusiavimas (sort)	0.018	0.017	0.018	0.018
	Skirstymas	0.037	0.033	0.034	0.034
	Viskas	0.505	0.509	0.514	0.509
1 000 000	Nuskaitymas	4.501	4.538	4.791	4.610
	Rusiavimas (sort)	0.467	0.459	0.460	0.462
	Skirstymas	0.410	0.413	0.410	0.411
	Viskas	5.378	5.409	5.662	5.483
10 000 000	Nuskaitymas	47.784	45.986	46.689	46.820

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
	Rusiavimas (sort)	8.143	8.123	8.132	8.133
	Skirstymas	5.014	4.936	4.968	4.973
	Viskas	60.942	59.045	59.790	59.926

1.6.3.3 STD::DEQUE

Ivesciu sk.	Matavimo rodmuo	Test 1 (s)	Test 2 (s)	Test 3 (s)	Vidurkis (s)
1 000	Nuskaitymas	0.005	0.006	0.006	0.006
	Rusiavimas (sort)	-	-	-	-
	Skirstymas	0.001	-	-	-
	Viskas	0.006	0.006	0.007	0.006
10 000	Nuskaitymas	0.047	0.046	0.046	0.046
	Rusiavimas (sort)	0.001	0.001	0.002	0.001
	Skirstymas	0.004	0.003	0.003	0.003
	Viskas	0.052	0.050	0.051	0.051
100 000	Nuskaitymas	0.450	0.453	0.452	0.452
	Rusiavimas (sort)	0.014	0.014	0.014	0.014
	Skirstymas	0.024	0.024	0.024	0.024
	Viskas	0.488	0.490	0.490	0.489
1 000 000	Nuskaitymas	4.506	4.536	4.520	4.521
	Rusiavimas (sort)	0.174	0.172	0.178	0.175
	Skirstymas	0.235	0.240	0.245	0.240
	Viskas	4.915	4.948	4.944	4.936
10 000 000	Nuskaitymas	45.918	45.904	48.004	46.609
	Rusiavimas (sort)	1.755	1.818	1.813	1.795
	Skirstymas	3.309	3.179	3.211	3.233
	Viskas	50.983	50.902	53.027	51.637

1.6.4 Tyrimo 1 rezultatu interpretacija

1.7 Testavimas - v1.0 optimizacija

Tyrimai buvo atlikti su Visual Studio 2022 /O2 optimizacijos nustatymais.
 Tyrimo 2 failai buvo sukurti viena karta prieš bandymu pradzia.
 Testavimo metu testavimo sistemoje nebuvo iijungtos jokios kitos programos.

1.7.1 Tyrimas 2 - Aprasas

Atliktas tyrimas su trejais skirtingais STL konteineriais (Vektoriai, Sarasai (list) ir Deque).
 Atlikti 3 bandymai kiekvienam konteineriui ir apskaiciuotas spartos vidurkis.

Kadangi spartos duomenys (skaiciai) buvo gan mazi, nebuvo apvalinta.

Siame tyrime buvo ismatuota minetu konteineriu sparta, programai skirstant duomenis pagal 4 strategijas:

1. Pirmajame tyrime naudota strategija - studentu konteinerio duomenų isvedimas i du naujus konteinerius naudojant `std::move()`, tuomet istrinimas tusciu duomenų is originalaus konteinerio;
2. Studentu konteinerio duomenų isvedimas i du naujus konteinerius naudojant `push_back()`;
3. Studentu konteinerio duomenų isvedimas i viena nauja ('blogu' studentu) konteineri naudojant `std::move()` ir tuo pat metu istrinimas is originalaus konteinerio, naudojant iteratorius;
4. 3 strategija, optimizuota naudojant `std::stable_partition()` ir `std::move()`.

1.7.2 Tyrimas 2 - Rezultatai

1.7.3 Tyrimo 2 rezultatu interpretacija

skyrius 2

Hierarchijos Indeksas

2.1 Klasių hierarchija

Šis paveldėjimo sąrašas yra beveik surikiuotas abėcėlės tvarka:

Timer	16
Zmogus	16
Studentas	15

skyrius 3

Klasės Indeksas

3.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

Studentas	15
Timer	16
Zmogus	16

skyrius 4

Failo Indeksas

4.1 Failai

Visų dokumentuotų failų sąrašas su trumpais aprašymais:

functions.h	19
mylib.h	19
Studentas.h	19
timer.h	24

skyrius 5

Klasės Dokumentacija

5.1 Studentas Klasė

Paveldimumo diagrama Studentas:

Bendradarbiavimo diagrama Studentas:

Vieši Metodai

- **Studentas** (std::string v, std::string p)
- **Studentas** (std::istream &in)
- **Studentas** (const [Studentas](#) &other)
- **Studentas** ([Studentas](#) &&other) noexcept
- [Studentas](#) & **operator=** (const [Studentas](#) &other)
- [Studentas](#) & **operator=** ([Studentas](#) &&other) noexcept
- const std::vector< double > & **getPazymiai** () const
- int **getPazymiaiSize** () const
- double **getEgzaminas** () const
- double **getGalutinis** (double*)(const std::vector< double > &)=calc_mediana) const
- std::istream & **read** (std::istream &input)
- void **addPazymys** (const double &pazymys)
- void **setGalutinis** (const double &galutinis)
- void **setEgzaminas** (const double &egzaminas)
- void **setRandVardas** ()
- void **setRandPavarde** (std::string &vardas)
- void **setRandEgzaminas** ()
- void **setRandPazymiai** ()
- void **paz_input** ()
- void **egz_input** ()

Vieši Metodai inherited from [Zmogus](#)

- **Zmogus** (std::string v, std::string p)
- **Zmogus** (std::istream &in)
- std::string **getVardas** () const
- std::string **getPavarde** () const
- void **setVardas** (std::string v)
- void **setPavarde** (std::string p)
- void **vardas_input** ()
- void **pavarde_input** ()

Draugai

- `std::istream & operator>>` (`std::istream &in`, [Studentas](#) &s)
- `std::ostream & operator<<` (`std::ostream &out`, const [Studentas](#) &s)

Additional Inherited Members

Apsaugoti Atributai inherited from [Zmogus](#)

- `std::string vardas_`
- `std::string pavarde_`

5.1.1 Metodų Dokumentacija

5.1.1.1 `read()`

```
std::istream & Studentas::read (  
    std::istream & input) [virtual]
```

Realizuoja [Zmogus](#).

Dokumentacija šiai klasei sugeneruota iš šių failų:

- `Studentas.h`
- `Studentas.cpp`

5.2 Timer Klasė

Vieši Metodai

- `void reset ()`
- `double elapsed () const`

Dokumentacija šiai klasei sugeneruota iš šio failo:

- `timer.h`

5.3 Zmogus Klasė

Paveldimumo diagrama Zmogus:

Vieši Metodai

- **Zmogus** (std::string v, std::string p)
- **Zmogus** (std::istream &in)
- std::string **getVardas** () const
- std::string **getPavarde** () const
- void **setVardas** (std::string v)
- void **setPavarde** (std::string p)
- virtual std::istream & **read** (std::istream &in)=0
- void **vardas_input** ()
- void **pavarde_input** ()

Apsaugoti Atributai

- std::string **vardas_**
- std::string **pavarde_**

Dokumentacija šiai klasei sugeneruota iš šių failų:

- Studentas.h
- Studentas.cpp

skyrius 6

Failo Dokumentacija

6.1 functions.h

```
00001 #pragma once
00002 #include "mylib.h"
00003
00004 void menu(int& choiceMenu);
00005
00006 int integer_input_validation(int lowEnd, int highEnd, std::string optionalPrompt = ""); //if highEnd =
    -1, no highEnd used
00007
00008 std::string string_input_validation(std::string optionalPrompt = "");
00009
00010 void student_file_generator(int nStud, int nPaz);
```

6.2 mylib.h

```
00001 #include <iostream>
00002 #include <iomanip>
00003 #include <string>
00004 #include <vector>
00005 #include <algorithm>
00006 #include <random>
00007 #include <stdlib.h>
00008 #include <fstream>
00009 #include <list>
00010 #include <deque>
00011 #include <sstream>
00012 #include <filesystem>
00013 #include "timer.h"
00014
00015 using std::cout;
00016 using std::cin;
00017
00018 namespace fs = std::filesystem;
```

6.3 Studentas.h

```
00001 #pragma once
00002 #include "mylib.h"
00003
00004 double calc_vidurkis(const std::vector<double>&); //Calculate vidurkis
00005 double calc_mediana(const std::vector<double>&); //Calculate mediana
00006
00007 class Zmogus {
00008 protected:
00009     std::string vardas_;
00010     std::string pavarde_;
00011 public:
00012     Zmogus() = default;
00013     Zmogus(std::string v, std::string p) : vardas_(v), pavarde_(p) {}
```

```

00014     Zmogus(std::istream& in) { in >> vardas_ >> pavarde_; }
00015     virtual ~Zmogus() = default;
00016
00017     inline std::string getVardas() const { return vardas_; }
00018     inline std::string getPavarde() const { return pavarde_; }
00019
00020     void setVardas(std::string v) { vardas_ = v; }
00021     void setPavarde(std::string p) { pavarde_ = p; }
00022
00023     virtual std::istream& read(std::istream& in) = 0;
00024
00025     void vardas_input();
00026     void pavarde_input();
00027 };
00028
00029 class Studentas : public Zmogus {
00030 private:
00031     std::vector<double> paz_; // nd pazymiai
00032     double egzaminas_ = 0;
00033     double galutinis_ = 0;
00034
00035 public:
00036     Studentas() = default;
00037     Studentas(std::string v, std::string p) : Zmogus(v, p) {}
00038     Studentas(std::istream& in);
00039     ~Studentas() override = default;
00040
00041     Studentas(const Studentas& other); //copy
00042     Studentas(Studentas&& other) noexcept; //move
00043
00044     Studentas& operator=(const Studentas& other); //copy assign
00045     Studentas& operator=(Studentas&& other) noexcept; //move assign
00046
00047     friend std::istream& operator>>(std::istream& in, Studentas& s); //input
00048     friend std::ostream& operator<<(std::ostream& out, const Studentas& s); //output
00049
00050     const std::vector<double>& getPazymiai() const { return paz_; };
00051     int getPazymiaiSize() const { return paz_.size(); };
00052     double getEgzaminas() const { return egzaminas_; };
00053     double getGalutinis(double (*) (const std::vector<double>&) = calc_mediana) const; //returns
00054     apdorotas galutinis
00055     std::istream& read(std::istream& input);
00056
00057     void addPazymys(const double& pazymys) { paz_.push_back(pazymys); };
00058     void setGalutinis(const double& galutinis) { galutinis_ = galutinis; }
00059     void setEgzaminas(const double& egzaminas) { egzaminas_ = egzaminas; };
00060
00061     void setRandVardas();
00062     void setRandPavarde(std::string& vardas);
00063     void setRandEgzaminas();
00064     void setRandPazymiai();
00065
00066     //Input funkcijos
00067     void paz_input();
00068     void egz_input();
00069 };
00070
00071 //Perskaityti egzistuojanti studentu duomenu faila ir irasyti i stl konteineri
00072 template<typename StudentaiContainer>
00073 void read_file(const std::string& filename, StudentaiContainer& studentai) {
00074     fs::path filePath = filename;
00075
00076     try {
00077         //CHECK IMPORTANT EXCEPTIONS
00078
00079         if (filePath.extension() != ".txt") {
00080             throw std::runtime_error("\n---KLAIDA: Failas " + filename + " turi baigtis '.txt'---\n");
00081         }
00082
00083         if (!fs::exists(filename)) {
00084             throw std::runtime_error("\n---KLAIDA: Failas " + filename + " neegzistuoja---\n");
00085         }
00086
00087         //Open file
00088         std::fstream fin(filename, std::ios::in);
00089
00090         if (!fin.is_open()) {
00091             throw std::runtime_error("\n---KLAIDA: Failo " + filename + " nepavyko atidaryti---\n");
00092         }
00093
00094         //Read file
00095         std::string curr_eil;
00096
00097         fin.ignore(INT32_MAX, '\n');
00098
00099         while (std::getline(fin, curr_eil)) {

```



```

00100         std::istringstream iss(curr_eil);
00101
00102         //read student
00103         Studentas temp_studentas(iss);
00104
00105         studentai.push_back(temp_studentas);
00106     }
00107
00108     fin.close();
00109 }
00110 catch (const std::exception& e) {
00111     std::cerr << e.what() << '\n';
00112 }
00113 }
00114
00115 //Irasyti studentu duomenis i faila / sukurti nauja faila su duomenimis
00116 template<typename StudentaiContainer>
00117 void write_studentai(const std::string& filename, StudentaiContainer& studentai) {
00118     //write to file
00119     std::ofstream fout(filename);
00120
00121     fout << '\n' << std::setw(15) << std::left << "Vardas" << std::setw(15) << std::left << "Pavarde"
00122         << std::setw(15) << std::left << "Galutinis (Vid.)" << std::setw(15) << std::left <<
00123         "Galutinis (Med.)"
00124         << std::setw(15) << std::left << "Egz." << '\n';
00125     for (const auto& s : studentai) {
00126         fout << s << '\n';
00127     }
00128
00129     fout.close();
00130 }
00131
00132 template<typename StudentaiContainer>
00133 void choice_sort(StudentaiContainer& studentai, int choice) {
00134     sort(studentai.begin(), studentai.end(),
00135         [choice](const Studentas& a, const Studentas& b) -> bool {
00136             if (choice == 1) {
00137                 if (a.getVardas() != b.getVardas()) return a.getVardas() < b.getVardas();
00138             }
00139             else if (choice == 2) {
00140                 if (a.getPavarde() != b.getPavarde()) return a.getPavarde() < b.getPavarde();
00141             }
00142             else if (choice == 3) {
00143                 return a.getGalutinis(calc_vidurkis) > b.getGalutinis(calc_vidurkis);
00144             }
00145             else {
00146                 return a.getGalutinis() > b.getGalutinis();
00147             }
00148         });
00149 }
00150
00151 //Sort vector
00152 void vidurkis_sort(std::vector<Studentas>& studentai);
00153
00154 //Sort deque
00155 void vidurkis_sort(std::deque<Studentas>& studentai);
00156
00157 //Sort list
00158 void vidurkis_sort(std::list<Studentas>& studentai);
00159
00160 void split_file_generator(const std::string& filename, std::vector<Studentas>& studentai);
00161
00162 void student_split(std::string filename, std::vector<Studentas>& studentai);
00163
00164 void testing_v04_1(int nStud);
00165
00166 void testing_v04_2(std::string filename);
00167
00168
00169
00170
00171
00172
00173 template <typename StudentaiContainer>
00174 void student_split_strategyold(StudentaiContainer& studentai) {
00175     std::vector<Studentas> studGeri, studBlogi;
00176     for (auto& s : studentai) {
00177         if (s.getGalutinis(calc_vidurkis) < 5.0)
00178             studBlogi.push_back(std::move(s));
00179         else
00180             studGeri.push_back(std::move(s));
00181     }
00182     studentai.clear();
00183 }
00184
00185 template <typename StudentaiContainer>
00186 void student_split_strategy1(StudentaiContainer& studentai) {
00187     StudentaiContainer studBlogi, studGeri;

```

```

00188     for (const auto& s : studentai) {
00189         if (s.getGalutinis(calc_vidurkis) < 5.0)
00190             studBlogi.push_back(s);
00191         else
00192             studGeri.push_back(s);
00193     }
00194 }
00195
00196 template <typename StudentaiContainer>
00197 void student_split_strategy2(StudentaiContainer& studentai) {
00198     StudentaiContainer studBlogi;
00199
00200     auto it = studentai.begin();
00201     while (it != studentai.end()) {
00202         if (it->getGalutinis(calc_vidurkis) < 5.0) {
00203             studBlogi.push_back(std::move(*it));
00204             //push iterator and remove studBlogi from studentai
00205             it = studentai.erase(it);
00206         }
00207         else {
00208             ++it;
00209         }
00210     }
00211 }
00212
00213 template <typename StudentaiContainer>
00214 void student_split_strategy3(StudentaiContainer& studentai) {
00215     //reorder - good first, bad after
00216     auto partition_point = std::stable_partition(
00217         studentai.begin(), studentai.end(),
00218         [](const Studentas& s) { return s.getGalutinis(calc_vidurkis) >= 5.0; }
00219     );
00220     //move
00221     StudentaiContainer studBlogi;
00222     for (auto it = partition_point; it != studentai.end(); ++it) {
00223         studBlogi.push_back(std::move(*it));
00224     }
00225 }
00226
00227 //Initial test of containers: reading, sorting + splitting students, writing
00228 template<typename StudentaiContainer>
00229 void test1_containers(int nStud) {
00230     std::string filename = "studentai" + std::to_string(nStud) + ".txt";
00231     cout << "\n---TESTAVIMAS SU " << nStud << " STUDENTU IVESCIU---\n\n";
00232
00233     //reading
00234     Timer t;
00235     StudentaiContainer studentai;
00236
00237     read_file<StudentaiContainer>(filename, studentai);
00238
00239     double read_t = t.elapsed();
00240     cout << "Duomenų nuskaitymas iš failo " << filename << " užtruko: " << read_t << " s\n\n";
00241     system("pause");
00242
00243     //sort by galutinisVid
00244     Timer t1;
00245     vidurkis_sort(studentai);
00246     double sort_t = t1.elapsed();
00247     cout << "\nStudentų sort() " << filename << " užtruko: " << sort_t << " s\n\n";
00248     system("pause");
00249
00250     //split students
00251     Timer t2;
00252     student_split_strategyold(studentai);
00253     double split_t = t2.elapsed();
00254     cout << "\nStudentų paskirstymas į 'gerus' ir 'blogus' " << filename << " užtruko: " << split_t << " s\n\n";
00255     system("pause");
00256
00257     //all results for nStud
00258
00259     cout << "\n\n---REZULTATAI SU " << nStud << " STUDENTU IVESCIU---\n\n";
00260     cout << "Nuskaitymas: " << read_t << " s\n\n";
00261     cout << "Rusavimas (sort): " << sort_t << " s\n\n";
00262     cout << "Skirstymas: " << split_t << " s\n\n";
00263     cout << "Viskas: " << (read_t + sort_t + split_t) << " s\n\n";
00264 }
00265
00266 //Test every num of students with one type of container
00267 template <typename StudentaiContainer>
00268 void run_test_containers() {
00269     test1_containers<StudentaiContainer>(1000);
00270     test1_containers<StudentaiContainer>(10000);
00271 }

```

```

00274     testl_containers<StudentaiContainer>(100000);
00275     testl_containers<StudentaiContainer>(1000000);
00276     testl_containers<StudentaiContainer>(10000000);
00277 }
00278
00279 //Test every num of students with one type of container
00280 template <typename StudentaiContainer>
00281 void testl_splitting(int nStud) {
00282     std::string filename = "studentai" + std::to_string(nStud) + ".txt";
00283
00284     //read
00285     StudentaiContainer studentai_original;
00286     read_file<StudentaiContainer>(filename, studentai_original);
00287
00288     cout << "\n---STRATEGIJA 0 (PIRMINE)---\n\n";
00289
00290     //copy and sort
00291     StudentaiContainer studentai0 = studentai_original;
00292     vidurkis_sort(studentai0);
00293
00294     //split students and time
00295     Timer t;
00296     student_split_strategyold(studentai0);
00297     double split_t = t.elapsed();
00298     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t << "
s\n\n";
00299     system("pause");
00300
00301     cout << "\n---STRATEGIJA 1---\n\n";
00302
00303     //copy and sort
00304     StudentaiContainer studentai1 = studentai_original;
00305     vidurkis_sort(studentai1);
00306
00307     //split students and time
00308     Timer t1;
00309     student_split_strategy1(studentai1);
00310     double split_t1 = t1.elapsed();
00311     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t1 << "
s\n\n";
00312     system("pause");
00313
00314     cout << "\n---STRATEGIJA 2---\n\n";
00315
00316     //copy and sort
00317     StudentaiContainer studentai2 = studentai_original;
00318     vidurkis_sort(studentai2);
00319
00320     //split students and time
00321     Timer t2;
00322     student_split_strategy2(studentai2);
00323     double split_t2 = t2.elapsed();
00324     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t2 << "
s\n\n";
00325     system("pause");
00326
00327     cout << "\n---STRATEGIJA 3---\n\n";
00328
00329     //copy and sort
00330     StudentaiContainer studentai3 = studentai_original;
00331     vidurkis_sort(studentai3);
00332
00333     //split students and time
00334     Timer t3;
00335     student_split_strategy3(studentai3);
00336     double split_t3 = t3.elapsed();
00337     cout << "\nStudentu paskirstymas i 'gerus' ir 'blogus' " << filename << " uztruko: " << split_t3 << "
s\n\n";
00338     system("pause");
00339     system("cls");
00340 }
00341
00342 template <typename StudentaiContainer>
00343 void run_test_splitting() {
00344     testl_splitting<StudentaiContainer>(1000);
00345     testl_splitting<StudentaiContainer>(10000);
00346     testl_splitting<StudentaiContainer>(100000);
00347     testl_splitting<StudentaiContainer>(1000000);
00348     testl_splitting<StudentaiContainer>(10000000);
00349 }
00350
00351 void testROF();

```

6.4 timer.h

```
00001 #pragma once
00002 #include <chrono>
00003 class Timer {
00004 private:
00005     std::chrono::time_point<std::chrono::high_resolution_clock> start;
00006 public:
00007     Timer() : start{ std::chrono::high_resolution_clock::now() } {}
00008     void reset() {
00009         start = std::chrono::high_resolution_clock::now();
00010     }
00011     double elapsed() const {
00012         return std::chrono::duration<double>(std::chrono::high_resolution_clock::now() -
00013         start).count();
00014     };
00015 }
```

Rodyklė

read

Studentas, [16](#)

Studentas, [15](#)

read, [16](#)

StudentuDuom - studentu duomenų apdorojimo aplikacija, [1](#)

Timer, [16](#)

Zmogus, [16](#)